In [3]:
```
import numpy as np
from tqdm import tqdm
from sklearn.linear_model import Lasso
from sklearn.linear_model import LassoCV
from sklearn.metrics import mean_squared_error
```

In [4]:
```
# Question 1.1
def generate(n, d, seed):
    np.random.seed(seed)
    X = np.random.randn(n, d)
    w = np.arange(d)
    w[::5] = 0
    epsilon = np.random.randn(n)
    y = X @ w + epsilon
    return X, y, w

X, y, w_true = generate(1000, 20, 42)
```

In [5]:
```
# Question 1.2
n, d, m = 400, 20, 100
lambda_values = np.logspace(-3, 1, 50)

true_errors_split = []
est_errors_split = []
true_errors_cv = []
est_errors_cv = []
nonzero_recovery_split = []
nonzero_recovery_cv = []

n_large = 10000
X_large, y_large, _ = generate(n_large, d, seed=0)

for seed in tqdm(range(m)):
    X, y, w_true = generate(n, d, seed)
    perm = np.random.permutation(n)
    X, y = X[perm], y[perm]

    # split
    train_X, val_X, test_X = X[:200], X[200:300], X[300:]
    train_y, val_y, test_y = y[:200], y[200:300], y[300:]

    # Train/Validation Split
    val_mse = []
    models = []
    for lam in lambda_values:
        model = Lasso(alpha=lam, max_iter=10000)
        model.fit(train_X, train_y)
        val_mse.append(mean_squared_error(val_y, model.predict(val_X)))
        models.append(model)

    best_idx = np.argmin(val_mse)
    best_model = models[best_idx]

    # True error on large dataset
    true_mse_split = mean_squared_error(y_large, best_model.predict(X_large))
    true_errors_split.append(true_mse_split)
    est_errors_split.append(val_mse[best_idx])
    nonzero_recovery_split.append(np.mean((np.abs(best_model.coef_) > 1e-6) == (np.abs(w_true) > 1e-6)))

    # 5-Fold Cross-Validation
    lasso_cv = LassoCV(alphas=lambda_values, cv=5, max_iter=10000)
    lasso_cv.fit(train_X, train_y)

    # True error on large dataset
    true_mse_cv = mean_squared_error(y_large, lasso_cv.predict(X_large))
    true_errors_cv.append(true_mse_cv)
    best_idx_cv = np.argmin(np.mean(lasso_cv.mse_path_, axis=1))
    est_errors_cv.append(np.mean(lasso_cv.mse_path_[best_idx_cv]))
    nonzero_recovery_cv.append(np.mean((np.abs(lasso_cv.coef_) > 1e-6) == (np.abs(w_true) > 1e-6)))

# Compare estimated vs. true errors
mse_est_vs_true_split = mean_squared_error(true_errors_split, est_errors_split)
mse_est_vs_true_cv = mean_squared_error(true_errors_cv, est_errors_cv)

print("MSE between true and estimated errors (Split):", mse_est_vs_true_split)
print("MSE between true and estimated errors (5-CV):", mse_est_vs_true_cv)
print("Avg correct nonzero recovery (Split):", np.mean(nonzero_recovery_split))
print("Avg correct nonzero recovery (5-CV):", np.mean(nonzero_recovery_cv))
```

```
  0%|          | 0/100 [00:00<?, ?it/s]
100%|██████████| 100/100 [00:11<00:00,  8.89it/s]
MSE between true and estimated errors (Split): 0.03715620277347146
MSE between true and estimated errors (5-CV): 0.017068685604093706
Avg correct nonzero recovery (Split): 0.8410000000000001
Avg correct nonzero recovery (5-CV): 0.8385000000000001
```

In [28]:
```
# Question 1.3
n, d, m = 400, 20, 100
lambda_values = np.logspace(-3, 1, 50)

true_errors_split = []
est_errors_split = []
true_errors_cv = []
est_errors_cv = []
nonzero_recovery_split = []
nonzero_recovery_cv = []

for seed in tqdm(range(m)):
    X, y, w_true = generate(n, d, seed)
    perm = np.random.permutation(n)
    X, y = X[perm], y[perm]

    # split
    train_X, val_X, test_X = X[:200], X[200:300], X[300:]
    train_y, val_y, test_y = y[:200], y[200:300], y[300:]

    # Train/Validation Split
    val_mse = []
    models = []
    for lam in lambda_values:
        model = Lasso(alpha=lam, max_iter=10000)
        model.fit(train_X, train_y)
        val_mse.append(mean_squared_error(val_y, model.predict(val_X)))
        models.append(model)

    best_idx = np.argmin(val_mse)
    best_model = models[best_idx]
    test_mse = mean_squared_error(test_y, best_model.predict(test_X))

    true_errors_split.append(test_mse)
    est_errors_split.append(val_mse[best_idx])
    nonzero_recovery_split.append(np.mean((best_model.coef_ != 0) == (w_true != 0)))

    # 5-Fold Cross-Validation
    lasso_cv = LassoCV(alphas=lambda_values, cv=5, max_iter=10000)
    lasso_cv.fit(train_X, train_y)
    test_mse_cv = mean_squared_error(test_y, lasso_cv.predict(test_X))

    true_errors_cv.append(test_mse_cv)
    best_idx_cv = np.argmin(np.mean(lasso_cv.mse_path_, axis=1))
    est_errors_cv.append(np.mean(lasso_cv.mse_path_[best_idx_cv]))
    nonzero_recovery_cv.append(np.mean((lasso_cv.coef_ != 0) == (w_true != 0)))

# Compare estimated vs. true errors
mse_est_vs_true_split = mean_squared_error(true_errors_split, est_errors_split)
mse_est_vs_true_cv = mean_squared_error(true_errors_cv, est_errors_cv)

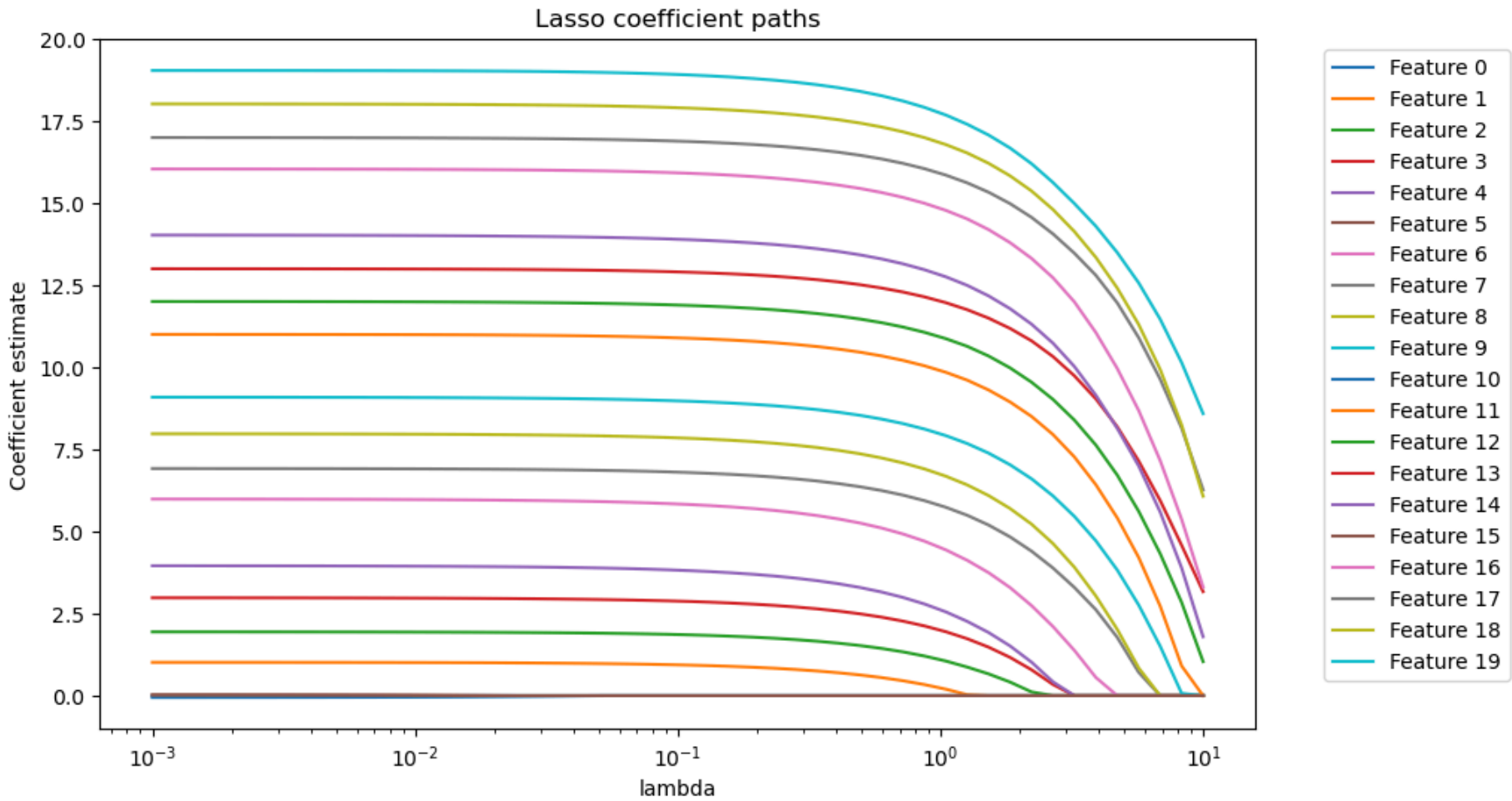
print("MSE between true and estimated errors (Split):", mse_est_vs_true_split)
print("MSE between true and estimated errors (5-CV):", mse_est_vs_true_cv)
print("Avg correct nonzero recovery (Split):", np.mean(nonzero_recovery_split))
print("Avg correct nonzero recovery (5-CV):", np.mean(nonzero_recovery_cv))
```

```
100%|██████████| 100/100 [00:08<00:00, 11.37it/s]
MSE between true and estimated errors (Split): 0.05027778570872459
MSE between true and estimated errors (5-CV): 0.03858883856964724
Avg correct nonzero recovery (Split): 0.8410000000000001
Avg correct nonzero recovery (5-CV): 0.8385000000000001
```

In [ ]:
```
# Question 1.3 cont.
import matplotlib.pyplot as plt

coefs = []
for lam in lambda_values:
    model = Lasso(alpha=lam, max_iter=10000)
    model.fit(X, y)  # fit on all data
    coefs.append(model.coef_)

coefs = np.array(coefs)
plt.figure(figsize=(10,6))
for i in range(d):
    plt.plot(lambda_values, coefs[:, i], label=f"Feature {i}")
plt.xscale('log')
plt.xlabel('lambda')
plt.ylabel('Coefficient estimate')
plt.title('Lasso coefficient paths')
plt.legend(bbox_to_anchor=(1.05,1), loc='upper left')
plt.show()
```


Lasso coefficient paths
Coefficient estimate
lambda
20.0
17.5
15.0
12.5
10.0
7.5
5.0
2.5
0.0
10^{-3}
10^{-2}
10^{-1}
10^{0}
10^{1}
Feature 0
Feature 1
Feature 2
Feature 3
Feature 4
Feature 5
Feature 6
Feature 7
Feature 8
Feature 9
Feature 10
Feature 11
Feature 12
Feature 13
Feature 14
Feature 15
Feature 16
Feature 17
Feature 18
Feature 19